# Composite Actions

Engenharia de CI/CD • Padronização e Reutilização Avançada no GitHub Actions

## 1. Fundamentos e Arquitetura Teórica

Em projetos corporativos e complexos, workflows de Integração Contínua (CI) frequentemente acumulam blocos repetitivos de comandos, como validações de código, execução de linters e testes unitários. Para evitar duplicação de código em múltiplos arquivos YAML, o GitHub Actions oferece as **Composite Actions**.

Uma Composite Action permite agrupar uma sequência estruturada de passos (*steps*) em um único componente modular encapsulado. O grande diferencial arquitetural desse recurso é que ele opera diretamente na camada de execução de passos, permitindo que a lógica modular seja injetada no meio de um job já existente sem alterar sua infraestrutura global.

**Modularidade de Passos**

Agrupa múltiplos comandos shell ou actions públicas em uma única chamada fluida e limpa.

**Escopo Local ou Global**

Pode ser hospedada no próprio repositório (caminho relativo) ou centralizada em um repositório dedicado de organização.

## 2. O Cenário Prático da Equipe

Imagine uma equipe de engenharia que precisa padronizar uma sequência rigorosa de validação. O fluxo de trabalho padrão de um job de build exige três fases sequenciais lineares: primeiro, o código-fonte é baixado para o runner; em seguida, executa-se a sequência de validações personalizadas (linter e testes); por fim, os relatórios e artefatos de compilação são enviados para o servidor.

**Objetivo:** Reutilizar exatamente a sequência intermediária de validação entre a etapa de checkout e a etapa de upload, mantendo todo o processo unificado no mesmo job.

## 3. Implementação Prática: Estruturando o Componente

Para criar uma ação composta, define-se um arquivo de configuração padronizado chamado `action.yml` dentro de uma pasta dedicada no repositório (ex: `.github/actions/validacao-padrao/`). A diretiva fundamental que caracteriza o comportamento composto é a propriedade `runs.using: composite`.

```
name: 'Validação Padrão de Projeto'
description: 'Executa linter e bateria de testes automatizados'
runs:
  using: "composite"
  steps:
    - name: Executar Analisador Estático (Linter)
      run: npm run lint
      shell: bash

    - name: Executar Testes Unitários
      run: npm test -- --coverage
      shell: bash
```

## 4. Consumindo a Ação no Workflow Principal

Com a composite action declarada, ela pode ser consumida nativamente dentro de qualquer job existente na lista de `steps` utilizando a diretiva `uses`, exatamente entre o passo de checkout inicial e o passo de upload final.

```
name: Pipeline Principal
on: [push]

jobs:
  build-e-testes:
    runs-on: ubuntu-latest
    steps:
      - name: Baixar Código Fonte
        uses: actions/checkout@v4

      - name: Injetar Validações Padronizadas
        uses: ./.github/actions/validacao-padrao

      - name: Enviar Relatórios de Build
        uses: actions/upload-artifact@v4
        with:
          name: relatorios-cobertura
          path: coverage/
```